

Tool Use y Function Calling

Módulo 10 · Sistemas Agénticos y Multi-Agente

Function calling, parallel tool calling, diseño de herramientas, tool hallucination y sandbox para código.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

Tool Use y Function Calling

Las herramientas (tools) son la forma en que los agentes LLM interactúan con el mundo externo. Sin herramientas, un agente solo puede razonar: con herramientas, puede buscar información actualizada, ejecutar código, consultar bases de datos, enviar emails, hacer llamadas a APIs externas, y completar tareas en sistemas empresariales. La calidad y el diseño del set de herramientas disponibles es uno de los factores más determinantes en la capacidad de un sistema agéntico.

Function calling es el mecanismo técnico que permite a los modelos LLM modernos (GPT-4, Claude, Gemini) expresar que necesitan ejecutar una función específica. En lugar de generar texto libre con el nombre de la herramienta y los argumentos (que requeriría parsing complejo y propenso a errores), el modelo genera un bloque estructurado JSON con el nombre de la función y los argumentos válidos según el schema definido por el desarrollador. El developer ejecuta la función y devuelve el resultado al modelo para que continúe su razonamiento.

El mecanismo de function calling

El ciclo de function calling

El ciclo de function calling funciona en cuatro pasos. El desarrollador define las herramientas con sus schemas JSON (nombre, descripción, parámetros con tipos y si son requeridos). El modelo recibe el mensaje del usuario + la lista de herramientas disponibles y decide si usa una herramienta. Si el modelo decide usar una herramienta, devuelve un mensaje especial (tool_use block en Anthropic, function_call en OpenAI) con el nombre de la función y los argumentos. El desarrollador ejecuta la función con esos argumentos y devuelve el resultado como tool_result al modelo. El modelo procesa el resultado y decide si continuar con otra herramienta o producir la respuesta final.

El diseño de buenas herramientas

Las herramientas bien diseñadas tienen: nombres descriptivos que el modelo puede entender (search_web, get_company_financials, send_email son mejores que tool1, fetch_data, action3). Descripciones claras de qué hace la herramienta, cuándo usarla, y qué devuelve. Schemas JSON precisos con tipos, restricciones, y ejemplos cuando son útiles. Comportamiento predecible: la misma herramienta con los mismos argumentos debe producir el mismo resultado (o documentar explícitamente si es no determinista).

Tool calling en paralelo

Los modelos modernos (GPT-4, Claude 3.5+) soportan llamadas paralelas a múltiples herramientas en un solo turno. En lugar de llamar primero a search_web y esperar el resultado para después llamar a get_financial_data, el modelo puede decidir llamar a ambas simultáneamente. Esto reduce significativamente la latencia de agentes que hacen múltiples búsquedas independientes. La implementación requiere que el desarrollador ejecute las

llamadas paralelas con asynco o threading y devuelva todos los resultados juntos.

SECCIÓN 3 · CÓMO FUNCIONA

Tool hallucination y cómo prevenirla

El tool hallucination ocurre cuando el modelo invoca una herramienta con argumentos incorrectos (por ejemplo, pasar una cadena de texto donde se espera un número), invoca una herramienta que no existe en el set disponible (confunde nombres de herramientas), o genera resultados inventados para una herramienta sin haberla llamado realmente. Las mitigaciones son: schemas JSON estrictos (el structured output del model verifica que los argumentos son del tipo correcto), validación de argumentos en el cliente antes de ejecutar la herramienta, y logging de todas las llamadas a herramientas con sus argumentos para auditoría.

Herramientas con estado vs sin estado

Las herramientas sin estado (stateless) producen siempre el mismo resultado con los mismos argumentos (búsqueda web, consulta a base de datos). Son las más simples de usar y de testear. Las herramientas con estado (stateful) modifican el estado del sistema externo y pueden tener efectos secundarios: enviar un email, crear un registro en la base de datos, publicar un mensaje en Slack. Las herramientas stateful requieren mayor cuidado: el agente no debe llamarlas repetidamente si ya las ejecutó, y deben tener mecanismos de rollback para los casos donde la acción fue incorrecta.

Sandbox para ejecución de código

Permitir que un agente ejecute código arbitrario es una de las capacidades más poderosas pero también más peligrosas. Un sandbox de código aísla la ejecución para evitar que el código generado por el LLM pueda dañar el sistema host: acceder a archivos del sistema, hacer llamadas de red no autorizadas, consumir recursos excesivos, o ejecutar código malicioso. Las opciones de sandbox son: contenedores Docker efímeros, microVMs (Firecracker), servicios de ejecución de código en la nube (E2B, Modal), o el intérprete de código integrado de la API de OpenAI.

SECCIÓN 4 · EJEMPLOS REALES

- Agente de análisis financiero con parallel tool calling: el modelo recibe la pregunta "¿Cómo está valorada la empresa X respecto a sus competidores en el sector Y?" y decide llamar en paralelo a: `get_company_financials("empresa_X")`, `get_sector_benchmark("sector_Y")`, y `search_news("empresa_X últimas noticias")`. Los tres resultados se devuelven simultáneamente y el modelo los usa para construir el análisis comparativo. La latencia total es igual a la de la llamada más lenta, no a la suma de las tres.
- Ejecución de código en sandbox para análisis de datos: el agente de análisis de datos puede escribir y ejecutar código Python en un sandbox E2B para: procesar datasets con pandas/numpy, generar visualizaciones con matplotlib, calcular estadísticas complejas, y ejecutar modelos de machine learning. El código se ejecuta en un contenedor aislado sin acceso al sistema del usuario.

- Tool use en agente de ITSM (Zendesk + Jira): el agente tiene herramientas para: buscar tickets similares en Zendesk, consultar el historial de incidentes en Jira, crear un ticket nuevo, asignarlo al equipo correcto, y notificar al usuario por email. Cada herramienta tiene un schema JSON estricto y los errores de validación se devuelven al agente con mensajes claros sobre qué corregir.

SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: Definir demasiadas herramientas. Un agente con 50 herramientas disponibles tiene dificultades para seleccionar la correcta: el modelo puede confundir herramientas similares o elegir la menos adecuada. La regla práctica: 5-15 herramientas por agente bien definidas son más efectivas que 50 herramientas ambiguamente definidas.

Error 2: Descripciones de herramientas demasiado vagas. Una herramienta llamada "fetch_data" con la descripción "obtiene datos" es inutilizable: el modelo no sabe cuándo usarla ni cómo. Las descripciones deben incluir: qué hace exactamente, cuándo usarla (vs cuándo no), qué devuelve, y los casos edge importantes.

Error 3: Permitir ejecución de código sin sandbox. Dar al agente una herramienta exec(code) sin sandbox es un riesgo de seguridad crítico. El código generado por el LLM puede hacer rm -rf /, exfiltrar datos, o consumir todos los recursos del sistema. Siempre usa un sandbox para ejecución de código arbitrario.

SECCIÓN 6 · APLICACIÓN PRÁCTICA

Para implementar function calling con la API de Anthropic: define las herramientas como lista de dicts con name, description, y input_schema (JSON Schema). Envía las herramientas en el parámetro tools de la llamada al API. Si la respuesta contiene un block de type tool_use, extrae el name y el input, ejecuta la función correspondiente, y devuelve el resultado como un message de role user con un block de type tool_result y el tool_use_id correspondiente. Continúa el bucle hasta que la respuesta no contenga tool_use blocks.

Template de descripción de herramienta de alta calidad: "Nombre: search_company_news. Descripción: Busca noticias recientes sobre una empresa específica usando su nombre o ticker. Úsala cuando necesites información actualizada sobre eventos recientes de una empresa (adquisiciones, resultados financieros, escándalos, cambios de liderazgo). No usar para información histórica anterior a 2020. Devuelve: lista de artículos con título, fecha, fuente, y resumen. Parameters: company_name (string, requerido): nombre o ticker de la empresa. num_results (integer, opcional, default=5): número de artículos a devolver (1-20)."

SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M9-T3 (APIs de IA): el function calling es la implementación técnica del tool use a través de la API.
- M10-T8 (Protocolos MCP/A2A): MCP estandariza cómo se definen y exponen las herramientas para agentes.
- M12 (Seguridad): el sandbox para ejecución de código es un control de seguridad crítico para agentes con acceso a ejecución de código.

Function calling permite a los modelos LLM invocar funciones externas de forma estructurada. El ciclo es: definir herramientas con schemas JSON → modelo decide qué herramienta usar → ejecutar la función → devolver el resultado → el modelo continúa. Las mejores prácticas de diseño de herramientas: nombres descriptivos, descripciones precisas con cuándo usar y qué devuelve, 5-15 herramientas por agente (no 50), schemas JSON estrictos. Parallel tool calling reduce la latencia en llamadas independientes. El sandbox es obligatorio para ejecución de código arbitrario (E2B, Modal, Docker). Tool hallucination se mitiga con schemas estrictos y validación en cliente.